// SYSTEM: ZUMO_SHIELD_V1.2 + UNO_R3 // N48.717 E10.319

REV.1.2 // 2026-04-28

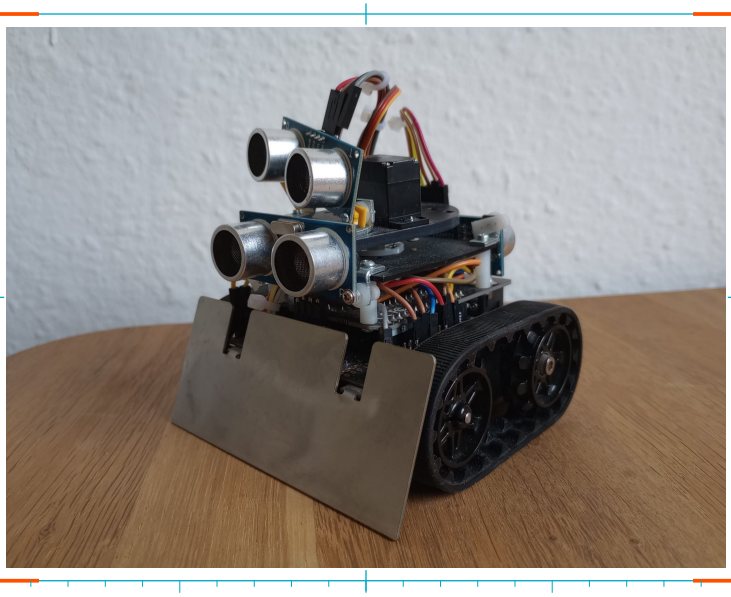
// PFLICHTENHEFT //

TSA E2 // 2025/26

ZUMO

SUMO-KAMPFROBOTER

// SUBJECT-04: ZUMO-SUMO // ● REC



// 2026.04.28 // KEMPTEN // TSA-E2 // V1.2

PROJEKT	Schaltungstechnik – Projektarbeit
SCHULE	Techniker Schule Allgäu – Kempten
KLASSE	Elektrotechnik E2 – Schuljahr 2025/26
TEAM	Henri Lechtenfeld
	Timo Dennis Görlich
BETREUUNG	Dr. Bernhard Fabel
HARDWARE	Pololu Zumo Shield V1.2 + Elegoo UNO R3
ZIEL	Autonomer Sumo-Roboter für 1m-Ring-Wettkampf

DOKUMENTVERSION 1.2 // STAND APRIL 2026

// INDEX //

INHALTSVERZEICHNIS

1	Projektbeschreibung
1.1	Aufgabenstellung
1.2	Ist-Zustand — Grundbausatz
1.3	Soll-Zustand — Endprodukt
2	Zielbestimmung
2.1	Musskriterien
2.2	Sollkriterien
2.3	Abgrenzungskriterien
3	Hardware-Konzept
3.1	Bauteile- und Sensorübersicht
3.2	Pin-Belegung
3.3	Timer-Belegung & Ressourcenkonflikte
4	Software-Konzept
4.1	Bibliotheken & Initialisierung
4.2	Zustandsmaschine (State Machine)
4.3	Sensorlogik
4.4	Servo-Turm — Timer2-ISR
5	Projektphasen & Zeitplan
6	Begleitende Präsentation
6.1	Konzept & Aufbau
6.2	Tech-Stack
7	Quellen
A	Anhang — Quellcode (CodePDF)

// CHAPTER 1 //

PROJEKTBESCHREIBUNG

§ 1.1

AUFGABENSTELLUNG

Im Schuljahr 2025/26 ist an der Technikerschule Allgäu in Kempten — Fachrichtung Elektrotechnik, Klasse E2 — im Fach Schaltungstechnik eine größere Projektarbeit zu erbringen, die als zweite Schulaufgabennote in die Halbjahresbewertung einfließt. Wir, Henri Lechtenfeld und Timo Dennis Görlich, haben uns für den Bau eines autonomen Sumo-Kampfroboters auf Basis des Pololu Zumo Shield V1.2 entschieden — ein Wettkampfroboter, der in einer kreisrunden Arena von etwa einem Meter Durchmesser gegen einen anderen Schülerroboter antritt. Ziel des Duells ist es, den Gegner über die durch ein weißes Klebeband markierte Ringgrenze zu drängen, ohne den Ring selbst zu verlassen.

Die Schule stellt drei baugleiche Zumo-Roboter bereit, die jedes Jahr von den Schülerinnen und Schülern für die Projektarbeit weiterentwickelt und vor der Übergabe an den nächsten Jahrgang aufbereitet werden — ein Roboter pro Gruppe. In unserem Jahrgang treten wir gegen eine zweite Gruppe an, die parallel an derselben Aufgabe arbeitet, und übernehmen einen Roboter, den unsere Vorgänger bereits um zusätzliche Sensorik erweitert hatten.

Da wir zuvor weder mit Arduino, Mikrocontroller-Programmierung noch mit Robotik gearbeitet hatten, war das Projekt für uns weitgehend Neuland. Neben der Entwicklung der eigentlichen Steuersoftware gehört auch eine eigenständig erstellte, interaktive Präsentation zum Pflichtumfang, die unser Konzept und unseren Aufbau am Abgabetag visuell und inhaltlich aufbereitet.

§ 1.2

IST-ZUSTAND — GRUNDBAUSATZ

Bei der Übergabe durch den Vorgänger-Jahrgang fanden wir einen aufgebauten und überarbeiteten Pololu Zumo vor. Im Kern besteht er aus dem **Pololu Zumo Shield V1.2** — einer Trägerplatine mit integrierter DRV8838-Motortreiberstufe, Buzzer, Reset-Taster und Batteriefach für vier AA-Mignon-Zellen — sowie einem aufgesteckten **Elegoo UNO R3**, einem Arduino-kompatiblen Board mit ATmega328P, das wir als Steuerrechner verwenden. Der Antrieb erfolgt über zwei Mikro-Getriebemotoren mit Kettenantrieb; vorne sitzt ein metallener Frontblade als Kontakt- und Schiebefläche.

An Sensorik bringt der Zumo bereits in seinem Grundzustand das **ZumoReflectanceSensorArray** mit fünf nach unten gerichteten Infrarot-Reflexionssensoren mit, mit denen sich die helle Ringgrenze gegen den dunklen Untergrund erkennen lässt. Auf dem Shield sind außerdem ein Gyroskop und ein Beschleunigungssensor verbaut, die wir über die Pololu-Bibliotheken auslesen und für gradgenaue Drehungen sowie für die Stoßerkennung nutzen.

Über den ursprünglichen Bausatz hinaus haben unsere Vorgänger den Roboter um **drei HC-SR04-Ultraschallsensoren** — vorne, hinten und auf einem drehbaren Servoturm — sowie einen **SG90-Servo** zur Turmrotation erweitert. Diese Aufbauten haben wir geerbt und in unserer Software weiterverwendet; mechanische Änderungen am Roboter selbst nehmen wir in dieser Iteration nicht vor.

§ 1.3

SOLL-ZUSTAND — ENDPRODUKT

Unser eigener Beitrag liegt vor allem in der Software. Aus dem weitgehend ungenutzten Bausatz unserer Vorgänger entwickeln wir eine vollständige autonome Steuerlogik, die alle vorhandenen Sensoren zu einem geschlossenen Wettkampfverhalten zusammenführt. Im Zentrum steht eine Zustandsmaschine mit acht Zuständen, die in jedem Moment festlegt, was der Roboter gerade tun soll — ob er den Gegner sucht, einen Treffer kontert, vor der Ringgrenze zurückweicht oder gerade ein Angriffsmanöver fährt.

Konkret zeigt sich das in mehreren Funktionsblöcken: Die Reflexionssensoren überwachen ständig die Ringgrenze und bringen den Roboter zuverlässig zum Wenden, bevor er den Ring verlässt. Die drei Ultraschallsensoren teilen sich die Sektoren — der Front-Sensor sucht nach Angriffszielen, der Heck-Sensor reagiert auf Angriffe von hinten, und der Turm-Sensor schwenkt kontinuierlich in einem 220°-Sektor und schließt damit die Sichtlücken der starr montierten Sensoren. Das Gyroskop liefert die Datengrundlage für gradgenaue Drehungen; 90°, 180° und kleinere Schwenks von 20° regeln wir über die integrierte Z-Achsen-Drehrate, statt uns auf ungenaue zeitbasierte Schätzungen zu verlassen. Schließlich registriert der Beschleunigungssensor einen seitlichen Stoß und löst ein sofortiges Konterangriffs-Verhalten aus.

// CHAPTER 2 //

ZIELBESTIMMUNG

§ 2.1

MUSSKRITERIEN

ID	Anforderung	Begründung
M1	Roboter passt in 10×10 cm Mini-Sumo-Klasse	Wettkampfregel
M2	Eigenständiger Betrieb ohne PC-Anbindung	Wettkampfregel
M3	Erkennt weiße Ringgrenze und verlässt Ring nicht	Wettkampfregel
M4	Detektiert gegnerischen Roboter im Frontbereich	Angriffsfähigkeit
M5	Reagiert auf Gegner mit Angriffsmanöver (CHARGE)	Wettkampfziel
M6	Start per Taster, 1 s Startverzögerung	Wettkampfregel
M7	Stromversorgung über 4×AA-Batterien	Bausatzvorgabe
M8	Quellcode kompiliert unter Arduino IDE	Toolchain

§ 2.2

SOLLKRITERIEN

ID	Anforderung	Begründung
S1	360°-Sicht durch drehbaren Sensorturm	Taktischer Vorteil
S2	Gradgenaue Drehungen über Gyro-Regelung	Zuverlässige Manöver
S3	Heck-Ultraschall — Reaktion auf Angriff von hinten	Defensiv-Vorteil
S4	Kollisionserkennung über Accelerometer	Trefferreaktion
S5	Richtungsabhängige Randerkennung (links/rechts)	Effizientere Wende
S6	Drallausgleich der Motoren	Geradeauslauf
S7	Modularer, abnehmbarer Aufbau (Pin-/Stiftleisten)	Wartbarkeit

§ 2.3

ABGRENZUNGSKRITERIEN

Folgende Funktionen sind **nicht** Bestandteil des Projekts:

- Bilderkennung über Kamera (Pixy o. Ä.) — Ersatz durch Ultraschall + Gyro
- Drahtlose Kommunikation oder Telemetrie zur Laufzeit
- Lithium-Akku-Betrieb — Stromversorgung bleibt bei 4×AA-Zellen
- Lernende oder adaptive Algorithmen (KI / ML)
- Steuerung über Fernbedienung — der Roboter operiert vollständig autonom

// CHAPTER 3 //

HARDWARE-KONZEPT

§ 3.1

BAUTEILE- UND SENSORÜBERSICHT

Komponente	Typ / Modell	Funktion	Anbindung
Mainboard	Elegoo UNO R3 (ATmega328P)	Steuerrechner	USB / Stiftleiste
Trägerplatine	Pololu Zumo Shield V1.2	Motortreiber, Sensorik, Power	Aufgesteckt
Antrieb	2× Mikro-Getriebemotoren	Fortbewegung über Kette	Über Shield (Timer1)
Bodensensoren	ZumoReflectanceSensorArray	Ringgrenzen-Erkennung	Digital, Pin 4 / 5
Ultraschall vorne	HC-SR04	Gegnerdetektion Front	Trig 3 / Echo A0
Ultraschall hinten	HC-SR04	Reaktion auf Heckangriff	Trig 11 / Echo A2
Ultraschall Turm	HC-SR04 (am Servoturm)	220°-Sweep, erweiterte Sicht	Trig 2 / Echo A3
Gyrosensor	L3G (I²C)	Gradgenaue Drehungen	I²C (SDA/SCL)
Accelerometer	LSM6DS33 (ZumoIMU)	Kollisionserkennung	I²C (SDA/SCL)
Servo	SG90 (270°-Variante)	Turmrotation	PWM-Pin 6 (Timer2)
Bedienelement	Zumo-Taster	Start / Stopp	ZUMO_BUTTON
Status-LED	On-Board-LED	Visuelle Rückmeldung	Pin 13
Versorgung	4× AA-Batterien (Mignon)	Energieversorgung	Über Shield-Batteriefach

§ 3.2

PIN-BELEGUNG

Pin	Richtung	Zugewiesen an	Bemerkung
D2	OUT	TRIG_PIN_TOWER	Ultraschall Turm
D3	OUT	TRIG_PIN_FRONT	Ultraschall Front
D4	IN	Reflectance Sensor links	QTR_THRESHOLD = 1800
D5	IN	Reflectance Sensor rechts	QTR_THRESHOLD = 1800
D6	OUT	SERVO_PIN	PWM via Timer2-ISR
D11	OUT	TRIG_PIN_REAR	Ultraschall Heck
D13	OUT	LED	On-Board-Status-LED
A0	IN	ECHO_PIN_FRONT	pulseIn() — 30 ms Timeout
A2	IN	ECHO_PIN_REAR	pulseIn() — 30 ms Timeout
A3	IN	ECHO_PIN_TOWER	pulseIn() — 30 ms Timeout
SDA / SCL	I/O	L3G Gyro + LSM6DS33 IMU	I²C-Bus, gemeinsam

§ 3.3

TIMER-BELEGUNG & RESSOURCENKONFLIKTE

Der ATmega328P stellt drei Hardware-Timer (Timer0/1/2) zur Verfügung. Die Belegung muss kollisionsfrei bleiben:

Timer	Belegung	Hinweis
Timer0	Arduino-Core: millis() / micros()	NICHT verändern
Timer1	ZumoMotors-Bibliothek (M1PWM/M2PWM)	Belegt — Servo.h verboten
Timer2	Servo-Steuerung via eigene ISR	Eigene Implementierung, 0,5 µs Auflösung

Hintergrund: Die Standard-`Servo.h`-Bibliothek würde Timer1 für sich beanspruchen und damit unsere Motorsteuerung blockieren. Wir erzeugen deshalb den Servo-PWM-Puls über eine eigene Timer2-ISR (Pulsbreite 500–2500 µs bei 50 Hz Wiederholrate) und behalten Timer1 vollständig den Motoren vor.

// CHAPTER 4 //

SOFTWARE-KONZEPT

§ 4.1

BIBLIOTHEKEN & INITIALISIERUNG

Eingebundene Bibliotheken:

- `Wire.h` — I²C-Kommunikation mit Gyro und IMU
- `ZumoMotors.h` — Motorsteuerung (Timer1-PWM, ±400 Geschwindigkeitsbereich)
- `ZumoShield.h` — Pushbutton, ZumoIMU (LSM6DS33), ReflectanceSensorArray
- `L3G.h` — Treiber für den zusätzlichen L3G-Gyrosensor

In unserem `setup()` initialisieren und kalibrieren wir alle Sensoren der Reihe nach. Gyro und Beschleunigungssensor mitteln wir über je 500 Messwerte, um den Nullpunkt-Offset (`gyroOffsetZ`, `accelOffsetX/Y/Z`) zu bestimmen — andernfalls würde sich schon ein kleiner Drift in den Drehregelkreis einschleichen. Anschließend starten wir die Servo-ISR und fahren den Turm in seine Mittelstellung. Danach wartet der Roboter auf den Tasterdruck und hält die vorgeschriebene Startverzögerung von einer Sekunde ein, bevor er losfährt.

§ 4.2

ZUSTANDSMASCHINE (STATE MACHINE)

Unsere Steuerlogik im `loop()` basiert auf einer `switch-case`-Zustandsmaschine mit den im Folgenden aufgeführten Zuständen. Im Lauf der Entwicklung haben wir bewusst auf zusätzliche Zustände wie einen MATADOR-Konter oder eine eigene DODGE-Routine verzichtet — was wir früh als „mehr Komplexität gleich besseres Verhalten“ eingeschätzt hatten, hat sich in den Tests als Fehlannahme erwiesen. Der schlanke Aufbau mit acht Zuständen reagiert im Ring verlässlicher als jede komplexere Variante, die wir testweise ausprobiert haben.

Zustand	Verhalten	Übergang
STATE_SEARCH	Vorwärtsfahrt mit FORWARD_SPEED=237	Bei Rand → REVERSE / bei Gegner → CHARGE
STATE_REVERSE	Rückwärtsfahrt mit REVERSE_SPEED=200, 300 ms	Nach Ablauf → TURN_LEFT/RIGHT
STATE_TURN_LEFT	Gyro-geregelte 90°-Linksdrehung	Nach Zielwinkel → SEARCH
STATE_TURN_RIGHT	Gyro-geregelte 90°-Rechtsdrehung	Nach Zielwinkel → SEARCH
STATE_SPIN	Gyro-geregelte 180°-Kehrtwende (Reaktion auf Heckangriff)	Nach Zielwinkel → CHARGE
STATE_CHARGE	Vollgas-Angriff mit FULL_SPEED=400, 500 ms	Nach Ablauf → SEARCH
STATE_SWIVEL_LEFT	20°-Schwenk nach links (Turm-Trefferreaktion)	Nach Ablauf → CHARGE
STATE_SWIVEL_RIGHT	20°-Schwenk nach rechts (Turm-Trefferreaktion)	Nach Ablauf → CHARGE

Den Zustandswechsel haben wir in einer Funktion `setState()` gekapselt. Sie setzt den neuen Zustand, speichert `stateStartTime = millis()` für zeitbasierte Übergänge und startet bei den Drehzuständen automatisch einen neuen Gyro-Zielwinkel (`gyroStartTurn(90 / 180 / 20)`) — so brauchen wir die Initialisierung nicht in jedem Case separat zu wiederholen.

§ 4.3

SENSORLOGIK

Ringgrenzen-Erkennung: Bei jedem Schleifendurchlauf vergleichen wir die beiden Reflectance-Sensoren (Pin 4/5) gegen den Schwellwert `QTR_THRESHOLD = 1800`. Wird einer der Sensoren unterschritten, wechselt der Roboter in `STATE_REVERSE`; die anschließende Wenderichtung richtet sich nach der Seite, auf der wir die Linie zuerst erkannt haben.

Gegnererkennung (Front): Unsere Funktion `distance_mm()` liefert die Entfernung in mm. Werte zwischen 0 und 300 mm im Frontkanal lösen `STATE_CHARGE` aus — der Roboter geht direkt in den Vollgas-Angriff.

Heckschutz: Werte unter 100 mm im Heckkanal triggern `STATE_SPIN` — eine 180°-Kehrtwende mit anschließendem CHARGE, sodass wir auf einen Angriff von hinten sofort kontern.

Turm-Sweep: Solange der Turm-Ultraschall *kein* Ziel sieht (distTower > 200 mm oder < 10 mm), drehen wir den Servo alle 10 ms um `turmSpeed = 10°` weiter (Bereich 80°–300°), bis er auf etwas trifft. Liegt das Ziel im Bereich 50–200 mm, wechseln wir je nach Turmwinkel in `STATE_SWIVEL_LEFT` oder `STATE_SWIVEL_RIGHT` — ein kurzer Schwenk, der den Roboter auf das Ziel ausrichtet, bevor der CHARGE einsetzt.

Gyro-Regelung: Die Z-Achsen-Drehrate skalieren wir mit `0,00875 °/s` pro LSB und integrieren sie über die Zeit. `gyroTurnDone()` beendet die Drehung, sobald wir den Zielwinkel erreicht haben; ein Sicherheits-Timeout von 2000 ms verhindert, dass der Roboter im Falle eines Sensorausfalls endlos rotiert.

Kollisionserkennung: Aus den drei Beschleunigungsachsen berechnen wir den Betrag ($\sqrt{ax^2+ay^2+az^2}$). Übersteigt dieser im SEARCH-Zustand 0,5 g, werten wir das als Treffer durch den Gegner und schalten direkt auf CHARGE — wir wollen den eingehenden Stoß nicht abwarten, sondern sofort zurückdrücken.

Drallausgleich: Unsere Funktion `drive()` kompensiert eine systematische Asymmetrie der Motoren bei Geradeausfahrt — der Roboter zog in unseren Tests merklich nach einer Seite. Bei Geschwindigkeiten ≤ 250 ziehen wir deshalb `TRIM_PERCENT_LOW = 4 %` vom linken Motor ab und schlagen sie auf den rechten auf.

§ 4.4

SERVO-TURM — TIMER2-ISR

Da Timer1 bereits durch die ZumoMotors-Bibliothek belegt ist, dürfen wir die Standard-`Servo.h` nicht verwenden — sie würde sich denselben Timer schnappen und unsere Motorsteuerung lahmlegen. Stattdessen erzeugen wir den PWM-Puls für den SG90-Servo manuell über eine eigene Timer2-Interrupt-Service-Routine:

- Timer2 läuft im CTC-Modus mit 1:8-Prescaler → 0,5 µs Zählschritt
- Die ISR `TIMER2_COMPA_vect` erzeugt 50 Hz Wiederholrate (20 ms Periode)
- Pulsbreite 500–2500 µs entspricht Winkel 0°–270°
- `servoSetPosition()` deaktiviert kurz den Interrupt, schreibt den neuen Sollwert und reaktiviert ihn — atomarer Zugriff

// CHAPTER 5 //

PROJEKTPHASEN & ZEITPLAN

Wir haben unsere Arbeit in fünf Wochenblöcke aufgeteilt, in denen sich Roboter-Entwicklung und begleitende Präsentation parallel weiterentwickelten. Da wir bei Arduino und Mikrocontroller-Programmierung Neuland betraten, war das erste Drittel überwiegend Einarbeitung; in den späteren Wochen verschob sich der Schwerpunkt auf Sensorintegration, Tests im Ring und Feinschliff. Die folgende Tabelle zeigt unsere Aufteilung im Detail:

Woche	Schwerpunkt	Roboter-Entwicklung	Präsentation
Woche 1	SETUP	■ Hardware, Arduino IDE & Schaltpläne gesichtet ■ OneDrive-Ordner als gemeinsame Arbeitsumgebung eingerichtet ■ Erste Zumo-Testprogramme — Inbetriebnahme der Motorsteuerung	—
Woche 2	STATE MACHINE	■ Zustandsmaschine als Kernlogik implementiert ■ Grundgerüst SEARCH / REVERSE / TURN / CHARGE	◆ Präsentation gestartet — Konzept & 3D-Modelle
Woche 3	SENSORIK I	■ Gyro-Logik — gradgenaue Drehungen über L3G ■ Ultraschall-Logik — Front, Heck und Turm ■ Servo-ISR über Timer2 entwickelt	◆ Präsentation — Hardware-Slides & Schaltpläne integriert
Woche 4	SENSORIK II & TESTING	■ Accelerometer — Kollisionserkennung ■ Pin-Belegung an Zumo Tower Schaltplan angepasst ■ Testing & Tuning im Ring mit Dummy-Roboter	◆ Präsentation — Code-Slides & Diagramme
Woche 5	FEINSCHLIFF	■ Finales Tuning der Schwellwerte und Drehparameter ■ Wettkampftests im Ring	◆ Präsentation fertiggestellt — finale HTML-Version V8

◆ Aufgaben rund um die begleitende Präsentation. ■ Aufgaben rund um die Roboter-Entwicklung.

// CHAPTER 6 //

BEGLEITENDE PRÄSENTATION

Neben dem Pflichtenheft fordert die Aufgabenstellung eine begleitende, interaktive Präsentation. Wir haben uns entschieden, diese als eigenständige Single-File-HTML-Anwendung zu bauen, statt auf vorgefertigte Slide-Tools wie PowerPoint oder Google Slides zurückzugreifen — die Präsentation ist damit unabhängig von Cloud-Diensten, läuft direkt im Browser und lässt sich als einzelne Datei dauerhaft archivieren. Sie dient sowohl als Live-Vortragsmedium am Abgabetag als auch als langfristiges Showcase-Format.

§ 6.1

KONZEPT & AUFBAU

- **Single-File HTML-Anwendung** — alle Assets (3D-Modelle, Audio, Logos, Schaltplan-PDFs) sind als Base64 direkt in einer einzigen `.html`-Datei eingebettet (ca. 24,6 MB).
- **17 Folien** — Hardware-Komponenten als 3D-Modelle (Slides 1–6), Zumo Tower Schaltplan-Popup (Slide 7), Code-Slides mit Syntax-Highlighting (Slides 8–14), interaktive Projekt-Timeline (Slide 15) und Credits (Slide 16–17).
- **3D-Modelle** einzeln pro Folie eingeblendet (Chassis, PCB, Arduino, Gyro, Motor, Sensor) mit Pivot-Gruppen-System und Erscheinungs-Animation.
- **FreeCam** — frei drehbare Kamera per Maus oder Touch-Geste (Orbit, Pan, Pinch-Zoom).
- **Schaltplan-Popups** — vier eingebettete PDFs (Reflectance-Array, Shield V1.2, Arduino UNO, HC-SR04, Zumo-Tower) mit pdf.js gerendert.
- **Responsive UI** — über 30 `clamp()`-Regeln machen die Präsentation auf 4K-Monitor wie auch Tablet darstellbar.

§ 6.2

TECH-STACK

Komponente	Verwendung
Three.js r128	3D-Rendering der Hardware-Modelle (GLB/GLTF)
GSAP 3.12.2	Folienübergänge, Erscheinungs-Animationen, Timeline
pdf.js 3.11.174	Rendering der Schaltplan-PDFs in Canvas-Elemente
GLTFLoader	Dynamisches Laden der 3D-Modelle aus Base64
Google Fonts	Bebas Neue, Share Tech Mono, Barlow Condensed
Build	Reines HTML/CSS/JS — kein Framework, kein Build-Step

Designkonzept: Visuell orientieren wir uns am „Vectorheart"-Stil — der Designsprache des Marathon-Reboots von Bungie aus dem Jahr 2026. Charakteristisch sind das dunkle Farbschema mit Cyan-, Yellow- und Orange-Akzenten, monospace-Typografie und eine an CCTV-Bildschirme angelehnte Anmutung. Auch dieses Pflichtenheft folgt — auf hellem Druck-Untergrund umgesetzt — derselben Designsprache. Der Quellcode der Präsentation ist über das in den Credits hinterlegte GitHub-Repository abrufbar.

// CHAPTER 7 //

QUELLEN

- Pololu — Zumo 32U4 Robot User's Guide / Zumo Shield User's Guide v1.2
- Arduino — Datenblatt ATmega328P (Microchip)
- STMicroelectronics — Datenblatt L3GD20 / LSM6DS33
- Datenblatt HC-SR04 Ultraschall-Distanzsensor
- TowerPro — Datenblatt SG90 Servo (270°-Variante)
- Three.js, GSAP, pdf.js — offizielle Dokumentationen (CDN-Versionen siehe Abschnitt 6.2)

// ANHANG A //
ZUMO_PROJEKTARBEIT_2026.INO

QUELL-CODE

ZUMO_PROJEKTARBEIT_2026.INO

Auf den folgenden Seiten ist der vollständige Arduino-Quellcode des Roboters abgedruckt. Die Originaldatei `Zumo_Projektarbeit_2026.ino` umfasst rund 490 Zeilen C++ und liegt im Projektordner bei.

```
// -------------------------------------------------------
// ZUMO SUMO-ROBOTER — Pololu Zumo Shield V1.3 + Elegoo UNO R3
// -------------------------------------------------------
//
// HARDWARE:
//   - Pololu Zumo Shield V1.3 mit Elegoo UNO R3 (Arduino-kompatibel)
//   - 2x Mikro-Getriebemotoren über ZumoMotors-Bibliothek
//   - 2x IR-Bodensensoren (ZumoReflectanceSensorArray, Pin 4 + 5)
//   - 3x HC-SR04 Ultraschallsensoren (vorne / hinten / Turm)
//   - 1x L3G Gyro-Sensor (I2C) für gradgenaue Drehungen
//   - 1x LSM6DS33 Accelerometer (I2C, via ZumoIMU) für Kollisionserkennung
//   - 1x SG90 Servo (Pin 6) für Turmdrehung — Timer2-ISR
//   - 1x Zumo-Taster zum Starten/Stoppen
//
// TIMER-BELEGUNG (ATmega328P):
//   Timer0 → millis(), micros() — Arduino-Core (NICHT ANFASSEN)
//   Timer1 → ZumoMotors (M1PWM/M2PWM) — BELEGT
//   Timer2 → Servo-ISR (NICHT ZumoBuzzer — Buzzer wird nicht verwendet)
//   Servo.h → VERBOTEN (würde Timer1 klauen → Konflikt mit ZumoMotors)
//
// ZUSTÄNDE:
//   SEARCH      — Fährt vorwärts, sucht den Gegner
//   REVERSE     — Rückwärts nach Randerkennung
//   TURN_LEFT   — Gyro-gesteuerte Linksdrehung (90°)
//   TURN_RIGHT  — Gyro-gesteuerte Rechtsdrehung (90°)
//   SPIN        — Gyro-gesteuerte 180°-Kehrtwende
//   CHARGE      — Vollgas vorwärts (Angriff, zeitgesteuert)
//   SWIVEL_LEFT - Gyro-gesteuerte Linksweiche (20°)
//   SWIVEL_RIGHT- Gyro-gesteuerte Rechtsweiche (20°)
//   DODGE       - Reaktion auf Kollisionserkennung
//   MATADOR     - Angriffsmaneuver
//
// AUTOREN: Henri Lechtenfeld, Timo Dennis Görlich
// SCHULE:  TSA — Elektrotechnik - E2
// -------------------------------------------------------

#include <Wire.h>
#include <ZumoMotors.h>
#include <ZumoShield.h>
#include <L3G.h>

// ============================================================
// PINS
// ============================================================

#define LED              13
#define SERVO_PIN         6

#define TRIG_PIN_FRONT    3
#define ECHO_PIN_FRONT   A0
#define TRIG_PIN_REAR    11
#define ECHO_PIN_REAR    A2
#define TRIG_PIN_TOWER    2
#define ECHO_PIN_TOWER   A3

// ============================================================
// KONSTANTEN
// ============================================================

#define QTR_THRESHOLD     1800
#define TRIM_PERCENT_LOW   4
#define TRIM_PERCENT_HIGH  0

#define REVERSE_SPEED    200
#define TURN_SPEED       400
#define FORWARD_SPEED    237
#define FULL_SPEED       400

#define REVERSE_DURATION 300     // ms
#define CHARGE_DURATION  500     // ms

#define RIGHT   1
#define LEFT   -1

// ============================================================
// OBJEKTE
// ============================================================

ZumoMotors motors;
Pushbutton button(ZUMO_BUTTON);

#define NUM_SENSORS 2
```

```
unsigned int sensor_values[NUM_SENSORS];
byte sensorPins[] = {4, 5};
ZumoReflectanceSensorArray sensors(sensorPins, NUM_SENSORS, 2000, 2);

// -----------Gyro
L3G   gyro;
float gyroOffsetZ    = 0;
float gyroAngle      = 0;
float gyroTarget     = 0;
unsigned long gyroLastMicros = 0;
bool  gyroTurnActive = false;

// -----------Accelerometer
ZumoIMU imu;
float accelOffsetX = 0;
float accelOffsetY = 0;
float accelOffsetZ = 0;

// ============================================================
// STATE MACHINE
// ============================================================

enum State
{
    STATE_SEARCH,
    STATE_REVERSE,
    STATE_TURN_LEFT,
    STATE_TURN_RIGHT,
    STATE_SPIN,
    STATE_CHARGE,
    STATE_SWIVEL_LEFT,
    STATE_SWIVEL_RIGHT
};

State         currentState   = STATE_SEARCH;
unsigned long stateStartTime = 0;
int           turnDirection  = RIGHT;
int           zigzagSide     = RIGHT;

// ============================================================
// SERVO — TIMER2-ISR
// ============================================================

uint16_t volatile servoTime     = 0;     // 0,5µs-Einheiten
uint16_t volatile servoHighTime = 3000;  // 1500µs default
boolean  volatile servoHigh     = false;

ISR(TIMER2_COMPA_vect)
{
    servoTime += OCR2A + 1;

    static uint16_t highTimeCopy  = 3000;
    static uint8_t  interruptCount = 0;

    if (servoHigh)
    {
        if (++interruptCount == 2)
        {OCR2A = 255;}
        if (servoTime >= highTimeCopy)
        {digitalWrite(SERVO_PIN, LOW);servoHigh=false;interruptCount = 0;}
    }
    else
    {
        if (servoTime >= 40000)
        {
            highTimeCopy = servoHighTime;
            digitalWrite(SERVO_PIN, HIGH);
            servoHigh      = true;
            servoTime      = 0;
            interruptCount = 0;
            OCR2A = ((highTimeCopy % 256) + 256) / 2 - 1;
        }
    }
}
```

```
void servoInit()
{
    digitalWrite(SERVO_PIN, LOW);
    pinMode(SERVO_PIN, OUTPUT);

    // Timer2 CTC-Modus, 1:8 Prescaler → 0,5µs Auflösung
    TCCR2A = (1 << WGM21);
    TCCR2B = (1 << CS21);
    TCNT2  = 0;
    OCR2A  = 255;
    TIMSK2 |= (1 << OCIE2A);
    sei();
}

// Pulsbreite setzen (in Mikrosekunden, z.B. 1000–2000)
void servoSetPosition(uint16_t highTimeMicroseconds)
{TIMSK2 &= ~(1 << OCIE2A);servoHighTime = highTimeMicroseconds * 2;TIMSK2 |= (1 << OCIE2A);}

// ============================================================
// TURM-SWEEP
// ============================================================

float         turmAngle      = 180.0;
int           turmDirection  = RIGHT;
float         turmSpeed      = 10.0;
unsigned long turmLastUpdate = 0;
const unsigned long turmInterval = 10;

// Winkel → Pulsbreite (µs) für 270°-Servo
uint16_t angleToMicros(float deg)
{
    float value = 500.0 + (deg / 270.0) * 2000.0;
    if (value < 500)  value = 500;
    if (value > 2500) value = 2500;
    return (uint16_t)value;
}

// ============================================================
// SETUP
// ============================================================

void setup()
{
    pinMode(LED, OUTPUT);
    waitForButtonAndCountDown();

    Serial.begin(9600);

    // -----------Gyro initialisieren (L3G)
    Wire.begin();
    if (!gyro.init())
    {
        Serial.println("Gyro nicht gefunden!");
    }
    gyro.enableDefault();
    Serial.println("Kalibriere Gyro...");

    long sum = 0;
    const int samples = 500;
    for (int i = 0; i < samples; i++)
    {
        gyro.read();
        sum += gyro.g.z;
        delay(0.5);
    }
    gyroOffsetZ = sum / (float)samples;

    // -----------Accelerometer initialisieren (LSM6DS33 via ZumoIMU)
    if (!imu.init())
    {Serial.println("IMU nicht gefunden!"); }
    imu.enableDefault();
    Serial.println("Kalibriere Accelerometer...");

    long sumX = 0, sumY = 0, sumZ = 0;
    for (int i = 0; i < samples; i++)
    {
        imu.read();
        sumX += imu.a.x;
        sumY += imu.a.y;
        sumZ += imu.a.z;
        delay(0.5);
```

```
    }
    accelOffsetX = sumX / (float)samples;
    accelOffsetY = sumY / (float)samples;
    accelOffsetZ = sumZ / (float)samples;

    // -----------Servo initialisieren (Timer2-ISR)
    servoInit();
    servoSetPosition(angleToMicros(turmAngle));
    Serial.println("Servo gestartet (Timer2-ISR).");

    // -----------Ultraschall-Pins
    pinMode(TRIG_PIN_FRONT, OUTPUT);
    pinMode(TRIG_PIN_REAR,  OUTPUT);
    pinMode(TRIG_PIN_TOWER, OUTPUT);
    pinMode(ECHO_PIN_FRONT, INPUT);
    pinMode(ECHO_PIN_REAR,  INPUT);
    pinMode(ECHO_PIN_TOWER, INPUT);

    Serial.println("Setup abgeschlossen.");
}

// ============================================================
// HAUPTROUTINE
// ============================================================

void loop()
{
    // -----------Knopfdruck im Betrieb → Neustart
    if (button.isPressed())
    {
        motors.setSpeeds(0, 0);
        button.waitForRelease();
        waitForButtonAndCountDown();
        setState(STATE_SEARCH);
    }

    // -----------Bodensensoren einlesen
    sensors.read(sensor_values);
    bool borderLeft  = sensor_values[0] < QTR_THRESHOLD;   // Pin 4
    bool borderRight = sensor_values[1] < QTR_THRESHOLD;   // Pin 5

    // -----------Randerkennung
    if ((borderLeft || borderRight) &&
      currentState != STATE_REVERSE &&
      currentState != STATE_TURN_LEFT &&
      currentState != STATE_TURN_RIGHT &&
      currentState != STATE_SPIN)
    {
        turnDirection = borderLeft ? RIGHT : LEFT;
        setState(STATE_REVERSE);
    }

    // -----------Ultraschall: Entfernungsmessung
    if (currentState == STATE_SEARCH ||
        currentState == STATE_CHARGE ||
        currentState == STATE_SWIVEL_RIGHT ||
        currentState == STATE_SWIVEL_LEFT)
    {
        float distFront = distance_mm(TRIG_PIN_FRONT, ECHO_PIN_FRONT);
        if (distFront > 0 && distFront < 300)
        {setState(STATE_CHARGE);}

        float distRear = distance_mm(TRIG_PIN_REAR, ECHO_PIN_REAR);
        if (distRear > 0 && distRear < 100)
        {turnDirection = RIGHT;setState(STATE_SPIN);}

        float distTower = distance_mm(TRIG_PIN_TOWER, ECHO_PIN_TOWER);
        if (distTower > 50 && distTower < 400)
        {
            if (turmAngle > 180) {setState(STATE_SWIVEL_LEFT);}
            if (turmAngle <= 180) {setState(STATE_SWIVEL_RIGHT);}
        }
    }

    // -----------Kollisionserkennung über Accelerometer
    imu.readAcc();
    float ax = (imu.a.x - accelOffsetX) / 16384.0;
    float ay = (imu.a.y - accelOffsetY) / 16384.0;
    float az = (imu.a.z - accelOffsetZ) / 16384.0;
    float accelSum = sqrt((ax * ax) + (ay * ay) + (az * az));
```

```
if (accelSum > 0.6 && (currentState == STATE_SEARCH))
{setState(STATE_CHARGE);}

// -----------Turm-Sweep
float distTower = distance_mm(TRIG_PIN_TOWER, ECHO_PIN_TOWER);
if ((distTower > 400)||(distTower < 50))
{
    unsigned long nowMs = millis();
    if ((nowMs - turmLastUpdate) >= turmInterval)
    {
        turmLastUpdate = nowMs;
        turmAngle += (turmDirection * turmSpeed);
        if (turmAngle >= 300.0)
        {turmAngle     = 300.0;turmDirection = LEFT;}
        else if (turmAngle <= 80.0)
        {turmAngle     = 80.0;turmDirection = RIGHT;}
        servoSetPosition(angleToMicros(turmAngle));
    }
}

// -----------STATE MACHINE
switch (currentState)
{
    case STATE_SEARCH:
        if(elapsed()<= 1000){drive(FORWARD_SPEED, 100);}
        else{drive(100,FORWARD_SPEED);}
        if(elapsed()>= 2000){setState(STATE_SEARCH);}
        break;

    case STATE_REVERSE:
        drive(-REVERSE_SPEED, -REVERSE_SPEED);
        if (elapsed() >= REVERSE_DURATION)
        {setState(turnDirection == RIGHT ? STATE_TURN_RIGHT : STATE_TURN_LEFT);}
        break;

    case STATE_TURN_RIGHT:
        gyroUpdate();
        drive(TURN_SPEED, -TURN_SPEED);
        if (gyroTurnDone() || elapsed() >= 2000)
        {gyroTurnActive = false;motors.setSpeeds(0, 0);setState(STATE_SEARCH);}
        break;

    case STATE_TURN_LEFT:
        gyroUpdate();
        drive(-TURN_SPEED, TURN_SPEED);
        if (gyroTurnDone() || elapsed() >= 2000)
        {gyroTurnActive = false;motors.setSpeeds(0, 0);setState(STATE_SEARCH);}
        break;

    case STATE_SPIN:
        gyroUpdate();
        if (turnDirection == RIGHT)
        {drive(TURN_SPEED, -TURN_SPEED);}
        else
        {drive(-TURN_SPEED, TURN_SPEED);}
        if (gyroTurnDone() || elapsed() >= 2000)
        {gyroTurnActive  = false;motors.setSpeeds(0, 0);setState(STATE_CHARGE);}
        break;

    case STATE_CHARGE:
        drive(FULL_SPEED, FULL_SPEED);
        if (elapsed() >= CHARGE_DURATION)
        {setState(STATE_SEARCH);}
        break;

    case STATE_SWIVEL_LEFT:
        gyroUpdate();
        drive(0, FULL_SPEED);
        if (elapsed() >= 100)
        {gyroTurnActive = false;setState(STATE_CHARGE);}
        break;

    case STATE_SWIVEL_RIGHT:
        gyroUpdate();
        drive(FULL_SPEED, 0);
        if (elapsed() >= 100)
        {gyroTurnActive = false;setState(STATE_CHARGE);}
        break;
```

```
        default:
            drive(FORWARD_SPEED, FORWARD_SPEED);
            currentState = STATE_SEARCH;
            break;
    }
}

// ============================================================
// FUNKTIONEN
// ============================================================

// -----------Button
void waitForButtonAndCountDown()
{
    digitalWrite(LED, HIGH);
    button.waitForButton();
    digitalWrite(LED, LOW);
    setState(STATE_SEARCH);
    delay(1000);                        // 1s Startpause (Wettkampfregel)
}

// -----------State Machine
void setState(State newState)
{
    currentState   = newState;
    stateStartTime = millis();

    if ((newState == STATE_TURN_RIGHT || newState == STATE_TURN_LEFT)
        && !gyroTurnActive)
    {gyroStartTurn(90);}
    if (newState == STATE_SPIN && !gyroTurnActive)
    {gyroStartTurn(180);}
    if ((newState == STATE_SWIVEL_RIGHT || newState == STATE_SWIVEL_LEFT)
        && !gyroTurnActive)
    {gyroStartTurn(20);}
}

unsigned long elapsed()
{return millis() - stateStartTime;}

// -----------Ultraschall
float distance_mm(int Trigger, int echo)
{
    digitalWrite(Trigger, LOW);
    delayMicroseconds(10);
    digitalWrite(Trigger, HIGH);
    delayMicroseconds(10);
    digitalWrite(Trigger, LOW);
    float duration = pulseIn(echo, HIGH, 30000);
    float distance = duration * 0.34 / 2.0;        // Schallgeschwindigkeit ~0,34 mm/µs
    return distance;
}

// -----------Gyro
void gyroUpdate()
{
    if (!gyroTurnActive) return;

    gyro.read();
    unsigned long now = micros();
    float dt          = (now - gyroLastMicros) / 1e6;
    gyroLastMicros    = now;

    float z_dps = (gyro.g.z - gyroOffsetZ) * 0.00875;
    gyroAngle  += z_dps * dt;
}

void gyroStartTurn(float degrees)
{
    gyroAngle      = 0;
    gyroTarget     = degrees;
    gyroLastMicros = micros();
    gyroTurnActive = true;
}
```

```
bool gyroTurnDone()
{return fabs(gyroAngle) >= fabs(gyroTarget);}

// -----------Drallausgleich
void drive(int left, int right)
{
    if ((left > 0 && right > 0) || (left < 0 && right < 0))
    {
        int absSpeed = abs(left);
        int trim     = (absSpeed <= 250) ? TRIM_PERCENT_LOW : TRIM_PERCENT_HIGH;
        left  = left  - (left  * trim / 100);
        right = right + (right * trim / 100);
    }
    motors.setSpeeds(left, right);
}

// ==========================================================
// ENDE
// ==========================================================
```